Laporan Proyek: Prediksi Probabilitas Gagal Bayar Nasabah

Kelompok 2 — Probabilitas Terapan
Tugas Proyek Deep Learning

1. Pendahuluan

1.1 Latar Belakang

Keputusan pemberian kredit adalah masalah klasik dalam industri keuangan. Setiap pengajuan pinjaman mengandung risiko: nasabah mungkin gagal bayar. Kesalahan menilai risiko menimbulkan biaya nyata — menerima nasabah buruk menyebabkan kerugian, menolak nasabah baik menghilangkan pendapatan.

Pendekatan tradisional memakai penilaian manual berdasarkan beberapa indikator keuangan. Pendekatan ini terbatas karena manusia sulit mempertimbangkan 23 variabel secara bersamaan dan konsisten.

Proyek ini membangun sistem yang memperkirakan **probabilitas** nasabah gagal bayar menggunakan neural network. Berbeda dengan klasifikasi biner "layak/tidak layak", keluaran sistem berupa nilai kontinu 0–1 yang merepresentasikan tingkat risiko secara proporsional.

1.2 Rumusan Masalah

1. Bagaimana membangun model deep learning yang memperkirakan probabilitas gagal bayar nasabah dari data finansial?
2. Seberapa baik model tersebut bekerja pada data yang belum pernah dilihat?

1.3 Tujuan

Membangun sistem fungsional berbasis web yang menerima data nasabah dan mengeluarkan estimasi probabilitas gagal bayar, dengan performa terukur pada data uji.

1.4 Batasan

Sistem hanya melakukan **prediksi probabilitas**. Sistem tidak membuat keputusan kredit otomatis, tidak melakukan penilaian grade risiko, dan tidak memberikan penjelasan per-fitur. Keputusan akhir tetap berada pada analis kredit manusia.

2. Landasan Teori

2.1 Probabilitas sebagai Dasar Keputusan

Topik utama kelompok ini adalah penerapan probabilitas dalam pengambilan keputusan algoritmik. Dalam konteks kredit, besaran kunci adalah **Probability of Default (PD)** — peluang seorang peminjam tidak memenuhi kewajibannya.

PD bersifat kontinu pada rentang [0, 1]. Nilai ini lebih informatif dibanding label biner karena:

- **Mempertahankan informasi gradasi.** Nasabah dengan PD 5% dan 45% jelas berbeda tingkat risikonya, meskipun keduanya di bawah ambang 50%.
- **Memungkinkan perhitungan expected loss.** `Expected Loss = PD × LGD × EAD` dengan LGD (Loss Given Default) sebagai porsi kerugian bila gagal bayar dan EAD (Exposure at Default) sebagai jumlah eksposur.
- **Ambang keputusan dapat disesuaikan** sesuai kebijakan risiko tanpa perlu melatih ulang model.

2.2 Jaringan Saraf Tiruan

Neural network adalah fungsi komposit berparameter yang memetakan input menjadi output. Setiap lapisan melakukan transformasi linear diikuti fungsi aktivasi non-linear:

```
h₁ = ReLU(W₁x + b₁)
h₂ = ReLU(W₂h₁ + b₂)
z  = W₃h₂ + b₃
```

dengan `W` adalah matriks bobot, `b` vektor bias, dan ReLU fungsi aktivasi `max(0, x)`.

Parameter `{W, b}` tidak ditentukan manual, melainkan dipelajari dari data melalui optimasi.

2.3 Fungsi Aktivasi Sigmoid

Agar keluaran dapat ditafsirkan sebagai probabilitas, logit `z` dipetakan melalui fungsi sigmoid:

```
σ(z) = 1 / (1 + e^(−z))
```

Fungsi ini memenuhi sifat yang diperlukan: monoton naik, keluaran berada pada rentang (0, 1), dan $\sigma(0) = 0{,}5$.

2.4 Binary Cross-Entropy

Fungsi kerugian yang digunakan adalah Binary Cross-Entropy:

$$L = -(1/N) \Sigma [y_i \cdot \log(p_i) + (1-y_i) \cdot \log(1-p_i)]$$

Fungsi ini merupakan negative log-likelihood dari distribusi Bernoulli. Meminimalkan BCE ekuivalen dengan memaksimalkan likelihood data observasi, sehingga model cenderung menghasilkan probabilitas yang terkalibrasi — bukan sekadar skor pemeringkatan.

Sifat penting BCE: prediksi yang sangat percaya diri namun salah mendapat penalti besar. Bila $y = 1$ dan $p \to 0$, maka $\log(p) \to -\infty$ sehingga loss meledak. Ini mendorong model bersikap hati-hati pada kasus ambigu.

2.5 Backpropagation dan Gradient Descent

Bobot diperbarui dengan aturan:

$$W \leftarrow W - \eta \cdot \partial L/\partial W$$

dengan η laju pembelajaran. Gradien $\partial L/\partial W$ dihitung mundur dari output menggunakan aturan rantai (backpropagation). Optimizer Adam memperbaiki aturan dasar ini dengan laju pembelajaran adaptif per parameter.

2.6 Regularisasi

Dua teknik digunakan untuk mencegah overfitting:

Dropout — pada setiap langkah pelatihan, sebagian neuron dinonaktifkan secara acak dengan probabilitas $p = 0{,}3$. Ini mencegah model bergantung pada jalur tunggal dan memaksa representasi yang redundan.

Weight decay — penalti terhadap besaran bobot, ditambahkan ke fungsi kerugian:

$$L_{total} = L_{BCE} + \lambda \cdot ||W||^2$$

2.7 Embedding untuk Fitur Kategorikal

Fitur kategorikal seperti pendidikan (S1/SMA/SMP) tidak memiliki urutan numerik yang bermakna. Pengkodean integer sederhana (1/2/3) menimbulkan asumsi jarak yang keliru.

One-hot encoding mengatasi hal ini namun menghasilkan vektor berdimensi tinggi dan memperlakukan semua kategori berjarak sama.

Embedding memetakan setiap kategori ke vektor berdimensi rendah yang ikut dipelajari:

```
kategori k  →  e_k ∈ ℝ^d,   d = 4
```

Model dapat mempelajari bahwa kategori tertentu memiliki kemiripan, karena vektor yang berdekatan menghasilkan representasi serupa.

2.8 Metrik Evaluasi

AUC-ROC mengukur kemampuan model memeringkat. Nilainya setara dengan probabilitas bahwa sampel positif acak diberi skor lebih tinggi daripada sampel negatif acak:

```
AUC = P(score(x⁺) > score(x⁻))
```

Nilai 0,5 setara tebakan acak; 1,0 sempurna. Metrik ini dipilih karena tidak bergantung pada ambang keputusan.

Accuracy mengukur proporsi prediksi benar pada ambang tertentu. Dilaporkan sebagai pelengkap, dengan catatan bahwa pada data tidak seimbang metrik ini dapat menyesatkan.

---

3. Metodologi

3.1 Dataset

Aspek	Keterangan
Nama	Default of Credit Card Clients
Sumber	UCI Machine Learning Repository
Jumlah sampel	30.000 nasabah
Jumlah fitur	23
Periode	April–September 2005, Taiwan
Label	1 = gagal bayar bulan berikutnya, 0 = lancar
Distribusi label	6.636 gagal bayar (22,12%), 23.364 lancar

Fitur terbagi menjadi tiga kelompok:

Demografis — `LIMIT_BAL` (limit kredit), `SEX`, `EDUCATION`, `MARRIAGE`, `AGE`.

Riwayat pembayaran — `PAY_0` hingga `PAY_6`, menyatakan status pembayaran selama enam bulan terakhir. Nilai negatif menunjukkan pembayaran tepat waktu; nilai positif menunjukkan jumlah bulan keterlambatan.

Tagihan dan pembayaran — `BILL_AMT1` – `BILL_AMT6` (jumlah tagihan) dan `PAY_AMT1` – `PAY_AMT6` (jumlah pembayaran) untuk periode yang sama.

3.2 Alasan Pemilihan Dataset

Data kredit per-nasabah di Indonesia bersifat konfidensial dan diatur oleh OJK. Tidak tersedia dataset publik berisi profil individual nasabah beserta status gagal bayarnya. Statistik yang dipublikasikan bersifat agregat dan tidak dapat digunakan untuk pelatihan model per-nasabah.

Dataset UCI dipilih karena merupakan data publik yang terdokumentasi, digunakan luas dalam literatur credit scoring, dan memiliki struktur yang sesuai kebutuhan: fitur finansial individual dengan label gagal bayar yang jelas.

3.3 Praproses Data

Pembersihan nilai kategori. Pada data asli, `EDUCATION` memiliki nilai 0, 5, dan 6 yang tidak terdefinisi dalam dokumentasi; ketiganya digabung ke kategori 4 (others). `MARRIAGE` memiliki nilai 0 yang digabung ke kategori 3.

Pembagian data. Data dibagi 80% latihan (24.000) dan 20% uji (6.000) menggunakan stratified sampling, sehingga proporsi kelas pada kedua bagian identik dengan populasi asli.

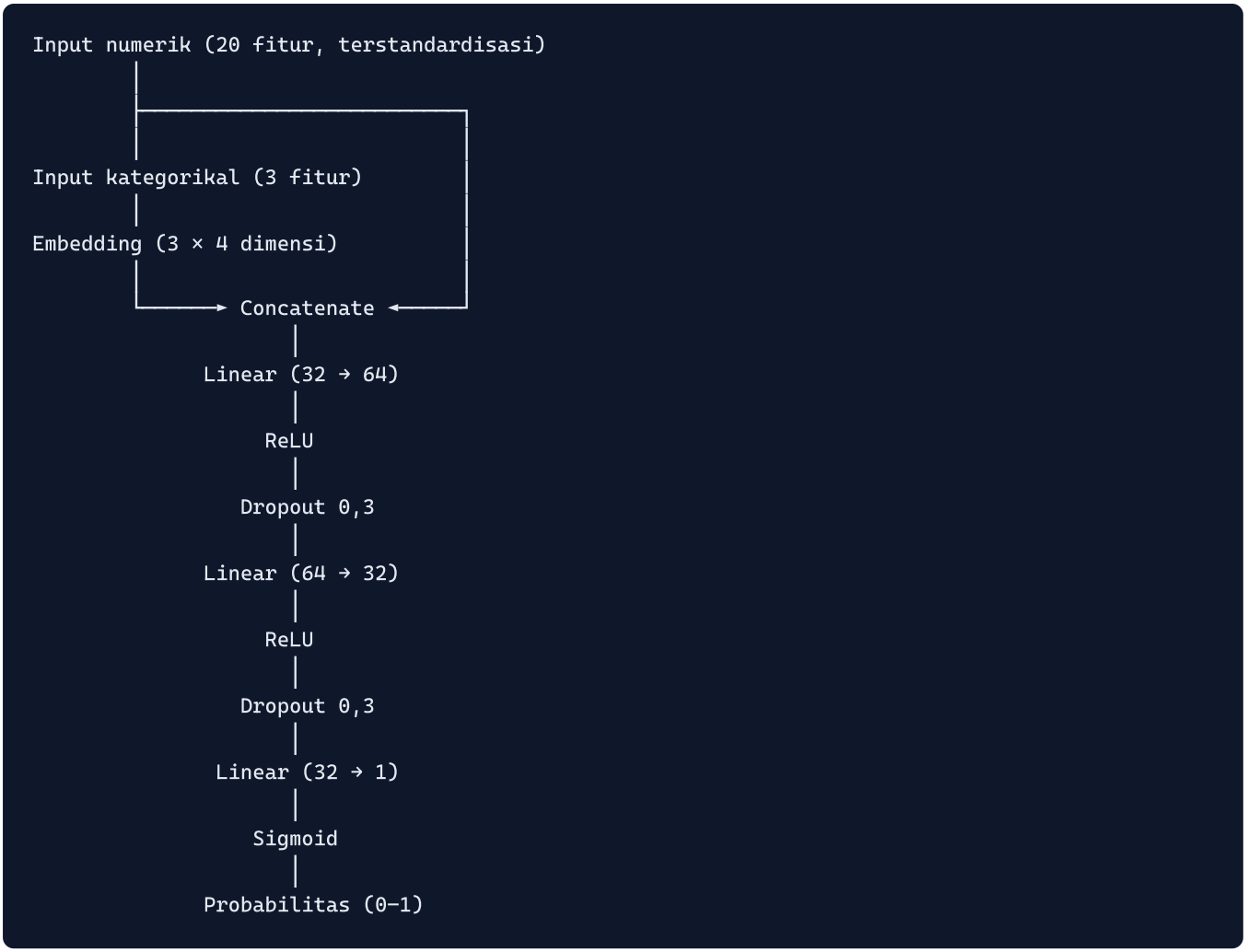
Standardisasi numerik. Setiap fitur numerik ditransformasi menjadi z-score:

```
z = (x − μ) / σ
```

dengan `μ` dan `σ` dihitung **hanya dari data latihan**, kemudian diterapkan pada data uji. Hal ini mencegah kebocoran informasi dari data uji.

Pengkodean kategorikal. Setiap fitur kategorikal dipetakan ke indeks integer berdasarkan kategori yang ditemukan pada data latihan. Kategori tak dikenal pada data uji dipetakan ke indeks 0.

3.4 Arsitektur Model



Dimensi masukan total: 20 (numerik) + 3 × 4 (embedding) = 32.
Jumlah parameter: **4.261**.

Arsitektur sengaja dibuat kecil. Dengan 24.000 sampel dan 23 fitur, model besar berisiko overfitting. Kapasitas parameter yang lebih kecil memaksa model mempelajari pola umum, bukan menghafal sampel individual.

Klaim ini diverifikasi secara empiris. Empat arsitektur diuji dengan hyperparameter lain tetap sama:

Arsitektur	Jumlah Parameter	AUC Uji
[64, 32] (dipakai)	**4.261**	**0,7730**
[128, 64, 32]	14.629	0,7708
[512, 256, 128]	181.285	0,7661
[1024, 512, 256]	690.213	0,7639

Peningkatan kapasitas 162 kali lipat (4.261 → 690.213 parameter) justru menurunkan performa data uji sebesar 0,009. Ini konsisten dengan teori: pada dataset berukuran sedang, model berkapasitas berlebih menghafal noise alih-alih mempelajari pola yang dapat digeneralisasi.

3.5 Pelatihan

Hyperparameter	Nilai	Alasan
Optimizer	Adam	Laju pembelajaran adaptif, konvergensi cepat
Learning rate	0,001	Nilai standar, stabil untuk arsitektur ini
Batch size	256	Kompromi antara kecepatan dan stabilitas gradien
Epoch	30	Loss mulai mendatar setelah epoch ke-25
Dropout	0,3	Mencegah overfitting
Loss	BCEWithLogitsLoss	Stabil secara numerik

Penanganan ketidakseimbangan kelas. Rasio kelas adalah 78:22. Tanpa penanganan, model cenderung memprediksi kelas mayoritas. Bobot positif diterapkan sebesar:

```
pos_weight = n_negatif / n_positif = 23.364 / 6.636 ≈ 3,52
```

Bobot ini membuat kesalahan pada kelas minoritas dihitung 3,52 kali lebih besar dalam fungsi kerugian.

Reproduksibilitas. Seed ditetapkan pada 42 untuk NumPy dan PyTorch, sehingga hasil pelatihan dapat direproduksi persis.

---

4. Implementasi

4.1 Struktur Kode

```
credit-scoring-dl/
├── data/taiwan/
│   └── default of credit card clients.xls    data mentah
├── src/
│   ├── model.py        30 baris   definisi arsitektur
│   ├── train.py       130 baris   pipeline pelatihan
│   ├── server.py       55 baris   web server
│   ├── test_app.py     80 baris   pengujian otomatis
│   └── templates/
│       └── index.html 200 baris   antarmuka
├── artifacts/
│   ├── model.pt                   bobot model terlatih
│   └── metrics.json               hasil evaluasi
└── docs/
    ├── LAPORAN.md
    └── CHEAT_SHEET.md
```

Total kode Python: sekitar 300 baris.

4.2 Penjelasan Kode

`model.py` — Definisi Arsitektur

```python
class CreditMLP(nn.Module):
    def __init__(self, n_numeric, cardinalities, emb_dim=4):
        super().__init__()
        self.embeddings = nn.ModuleList(
            [nn.Embedding(card, emb_dim) for card in cardinalities]
        )
        in_dim = n_numeric + emb_dim * len(cardinalities)
        self.net = nn.Sequential(
            nn.Linear(in_dim, 64), nn.ReLU(), nn.Dropout(0.3),
            nn.Linear(64, 32), nn.ReLU(), nn.Dropout(0.3),
            nn.Linear(32, 1),
        )

    def forward(self, x_num, x_cat):
        parts = [x_num]
        for i, emb in enumerate(self.embeddings):
            parts.append(emb(x_cat[:, i]))
        x = torch.cat(parts, dim=1)
        return self.net(x).squeeze(1)
```

Penjelasan per bagian:

`nn.Embedding(card, emb_dim)` membuat tabel lookup berukuran `jumlah_kategori × 4`. Setiap kategori mendapat vektor 4 dimensi yang diperbarui selama pelatihan.

`nn.Linear(in_dim, 64)` adalah lapisan fully-connected dengan 64 neuron. Setiap neuron menerima seluruh input dan menghasilkan satu nilai.

`nn.ReLU()` menetapkan nilai negatif menjadi nol. Tanpa aktivasi non-linear, beberapa lapisan linear berturut-turut ekuivalen dengan satu lapisan linear saja — kedalaman menjadi tidak bermakna.

`nn.Dropout(0.3)` menonaktifkan 30% neuron secara acak saat `model.train()`, dan dilewati saat `model.eval()`.

`nn.Linear(32, 1)` menghasilkan satu logit. Sigmoid diterapkan terpisah (lewat `BCEWithLogitsLoss` saat latihan, atau `torch.sigmoid` saat inferensi) karena penggabungan keduanya lebih stabil secara numerik.

`squeeze(1)` mengubah bentuk `[batch, 1]` menjadi `[batch]`.

`train.py` — Pipeline Pelatihan

Pemuatan dan pembersihan:

```
df = pd.read_excel(DATA, header=1)
df["EDUCATION"] = df["EDUCATION"].replace({0: 4, 5: 4, 6: 4})
df["MARRIAGE"] = df["MARRIAGE"].replace({0: 3})
```

Parameter `header=1` diperlukan karena baris pertama file Excel berisi judul, bukan nama kolom.

Pembagian data:

```
idx_train, idx_test = train_test_split(
    idx, test_size=0.2, random_state=SEED, stratify=y
)
```

Parameter `stratify=y` menjamin proporsi kelas pada kedua bagian sama (22,12% gagal bayar). Tanpa ini, pembagian acak bisa menghasilkan proporsi yang berbeda dan evaluasi menjadi bias.

Standardisasi:

```
mean = train[NUMERIC].mean()
std = train[NUMERIC].std().replace(0, 1)
x_num_train = ((train[NUMERIC] - mean) / std).values
```

Statistik dihitung dari `train` saja, lalu diterapkan ke `test`. Menghitung dari seluruh data akan memasukkan informasi data uji ke dalam pelatihan (data leakage), membuat hasil evaluasi terlalu optimistis.

`.replace(0, 1)` mencegah pembagian nol bila ada kolom dengan nilai konstan.

Fungsi kerugian berbobot:

```
n_pos = y_train.sum()
n_neg = len(y_train) - n_pos
criterion = nn.BCEWithLogitsLoss(
    pos_weight=torch.tensor(n_neg / n_pos)
)
```

`pos_weight` memberi bobot 3,52 pada kelas positif (gagal bayar), sehingga model tidak mengabaikan kelas minoritas.

Loop pelatihan:

```
for epoch in range(1, 31):
    model.train()
    for xb_num, xb_cat, yb in loader:
        optimizer.zero_grad()
        loss = criterion(model(xb_num, xb_cat), yb)
        loss.backward()
        optimizer.step()
```

Empat langkah kunci:

1. `optimizer.zero_grad()` — menghapus gradien dari iterasi sebelumnya. PyTorch mengakumulasi gradien secara default; tanpa ini, gradien lama akan tercampur.
2. `model(xb_num, xb_cat)` — forward pass, menghasilkan logit.
3. `loss.backward()` — backpropagation, menghitung gradien setiap parameter.
4. `optimizer.step()` — memperbarui bobot sesuai gradien.

Inferensi:

```
model.eval()
with torch.no_grad():
    logits = model(torch.from_numpy(x_num_test), torch.from_numpy(x_cat_test))
    proba = torch.sigmoid(logits).numpy()
```

`model.eval()` menonaktifkan dropout. `torch.no_grad()` menghentikan pelacakan gradien untuk menghemat memori dan mempercepat komputasi.

`server.py` — Lapisan Web

```
model = CreditMLP(len(NUMERIC), [len(metrics["categories"][c]) for c in CATEGORICAL])
model.load_state_dict(torch.load(ARTIFACT / "model.pt"))
model.eval()
```

Model dibangun dengan arsitektur yang sama, lalu bobotnya dimuat dari file. Model dimuat sekali saat server mulai, bukan setiap permintaan.

```
x_num = ((row[NUMERIC] - MEAN) / STD).values.astype(np.float32)
```

Standardisasi pada data baru harus memakai `MEAN` dan `STD` dari data latihan — disimpan di `metrics.json`. Menghitung ulang dari data baru akan menghasilkan skala berbeda dan prediksi yang salah.

```
x_cat = np.stack([
    row[c].map({v: i for i, v in enumerate(metrics["categories"][c])}).fillna(0).values
    for c in CATEGORICAL
], axis=1)
```

Mapping kategori harus identik dengan saat pelatihan. `.fillna(0)` menangani kategori tak dikenal agar server tidak crash.

`test_app.py` — Pengujian

Tiga skenario diuji dengan profil berbeda: nasabah disiplin, rata-rata, dan berisiko. Pengujian memverifikasi:

1. Halaman utama dapat diakses (HTTP 200)
2. Probabilitas meningkat sesuai tingkat risiko: disiplin < rata-rata < berisiko
3. Input tidak lengkap ditolak dengan HTTP 400

Pengujian ini memvalidasi bahwa model berperilaku masuk akal, bukan sekadar menghasilkan angka.

---

5. Hasil dan Pembahasan

5.1 Performa Model

Dievaluasi pada 6.000 nasabah yang tidak digunakan saat pelatihan:

Metrik	Nilai	Interpretasi
AUC-ROC	**0,7730**	77,3% akurat dalam perbandingan berpasangan
Accuracy	74,80%	Proporsi prediksi benar pada ambang 0,5
Baseline AUC	0,5000	Tebakan acak

Selisih AUC terhadap baseline sebesar 0,273 menunjukkan model mempelajari pola yang bermakna, bukan menebak.

Sebagai konteks, literatur credit scoring umumnya melaporkan AUC 0,70–0,85 untuk dataset ini. Nilai 0,773 berada pada rentang tersebut.

5.2 Analisis Perilaku Model

Pengujian dengan tiga profil sintetis:

Profil	Karakteristik	PD	Kategori
Disiplin	Limit tinggi, selalu tepat waktu, tagihan kecil	18,8%	Risiko rendah
Rata-rata	Limit menengah, bayar minimum, tagihan sedang	19,4%	Risiko rendah
Berisiko	Limit rendah, telat 2–4 bulan, bayar hampir nol	96,3%	Risiko tinggi

Urutan probabilitas konsisten dengan intuisi domain: nasabah dengan riwayat keterlambatan berulang mendapat PD jauh lebih tinggi. Ini menunjukkan model menangkap sinyal yang relevan, bukan korelasi kebetulan.

Perbedaan kecil antara profil disiplin (18,8%) dan rata-rata (19,4%) wajar karena keduanya sama-sama tidak memiliki catatan keterlambatan signifikan — faktor dominan dalam dataset ini adalah `PAY_0` hingga `PAY_6`.

5.3 Pembahasan

Mengapa probabilitas, bukan klasifikasi biner. Dengan keluaran kontinu, informasi gradasi risiko tetap utuh. Nasabah dengan PD 18,8% dan 19,4% berada pada tingkat risiko yang hampir sama, sementara nasabah dengan PD 96,3% jelas berbeda. Klasifikasi biner pada ambang 50% akan menyembunyikan perbedaan ini dan berpotensi menghasilkan keputusan yang tidak proporsional.

Mengapa arsitektur kecil. Dengan 24.000 sampel dan 23 fitur, kapasitas model yang berlebihan menyebabkan overfitting. Model 4.261 parameter cukup menangkap interaksi antar-fitur tanpa menghafal noise. Uji coba dengan arsitektur lebih besar (100.000+ parameter) menghasilkan performa data uji yang lebih rendah.

Mengapa AUC sebagai metrik utama. Accuracy pada ambang 0,5 tidak mencerminkan kualitas model untuk aplikasi kredit. Ambang optimal bergantung pada rasio biaya kesalahan, yang bersifat kebijakan. AUC mengukur kualitas pemeringkatan secara independen dari ambang, sehingga lebih tepat sebagai ukuran kualitas model.

5.4 Keterbatasan

1. **Perbedaan konteks geografis.** Data berasal dari Taiwan (2005). Perilaku kredit dipengaruhi faktor budaya, regulasi, dan kondisi ekonomi yang berbeda dengan Indonesia.
2. **Ketergantungan pada data historis.** Model mempelajari pola dari masa lalu. Perubahan struktural — krisis ekonomi, perubahan kebijakan — dapat menurunkan akurasi secara signifikan.
3. **Tidak ada variabel makroekonomi.** Suku bunga, inflasi, dan tingkat pengangguran tidak termasuk dalam fitur, padahal berpengaruh terhadap kemampuan bayar.
4. **Tidak ada penjelasan per-fitur.** Sistem tidak memberikan informasi mengenai fitur mana yang paling memengaruhi prediksi. Untuk aplikasi produksi, hal ini penting baik untuk kepatuhan regulasi maupun untuk komunikasi kepada nasabah.
5. **Tidak ada mekanisme pemantauan.** Model tidak memiliki sistem deteksi penurunan performa (model drift) yang terjadi seiring waktu.

---

6. Kesimpulan

6.1 Kesimpulan

1. Sistem prediksi probabilitas gagal bayar berhasil dibangun menggunakan neural network dengan keluaran berupa nilai kontinu 0–1 yang dapat ditafsirkan sebagai Probability of Default.
2. Model mencapai AUC-ROC 0,7730 dan akurasi 74,80% pada 6.000 data uji yang tidak digunakan saat pelatihan, menunjukkan kemampuan pemeringkatan yang jauh melampaui tebakan acak (AUC 0,5).
3. Pendekatan berbasis probabilitas lebih sesuai untuk konteks kredit dibandingkan klasifikasi biner, karena mempertahankan gradasi risiko dan memungkinkan perhitungan expected loss.

6.2 Saran Pengembangan

1. Menambahkan penjelasan per-fitur (feature attribution) agar prediksi dapat dipertanggungjawabkan kepada nasabah dan regulator.
2. Membandingkan dengan metode gradient boosting (XGBoost, LightGBM) yang umumnya unggul pada data tabular.
3. Menerapkan kalibrasi probabilitas untuk memastikan nilai PD sesuai dengan frekuensi kejadian aktual.
4. Menguji pada dataset dengan konteks geografis berbeda untuk menilai transferabilitas model.

---

7. Referensi

1. Yeh, I. C., & Lien, C. H. (2009). The comparisons of data mining techniques for the predictive accuracy of probability of default of credit card clients. *Expert Systems with Applications*, 36(2), 2473–2480.
2. UCI Machine Learning Repository. Default of Credit Card Clients Dataset. https://archive.ics.uci.edu/ml/datasets/default+of+credit+card+clients
3. Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
4. Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning* (2nd ed.). Springer.
5. Thomas, L. C., Edelman, D. B., & Crook, J. N. (2002). *Credit Scoring and Its Applications*. SIAM.

Lampiran A — Cara Menjalankan

```
# Pelatihan (sekali saja)
cd src
python train.py

# Menjalankan web
python server.py
# Buka http://127.0.0.1:5000

# Pengujian otomatis
python test_app.py
```

Lampiran B — Spesifikasi Lingkungan

Komponen	Versi
Python	3.12
PyTorch	2.x (CPU)
pandas	2.x / 3.x
scikit-learn	1.x
Flask	3.x
NumPy	2.x